Docker Build Cache Issue SOP

Objective

Resolve incorrect Docker build outputs caused by stale cached layers, reused dependencies, or outdated build artifacts.

Scope

Applicable to Docker-based services, containerized application deployments, DevOps teams, SRE teams, platform teams, and synthetic incident workflows across development, staging, and production-like environments.

Pre-requisites

1. Access to Docker host, Docker CLI, container runtime logs, and the target container or image name.
2. Permission to inspect containers, images, volumes, networks, resource limits, and compose files where applicable.
3. Access to application logs, deployment artifacts, Dockerfile, environment variables, and registry credentials.
4. Monitoring access for host CPU, memory, disk, container restart count, and container health checks.
5. Escalation contact for application, DevOps, security, registry, infrastructure, and platform teams.

Incident Metadata

Field	Value
Ticket ID	T024
Issue	Docker build cache issue
Category	Build
Service	Docker
Severity	Low
Status	Resolved
Description	Old layers reused incorrectly
Expected Resolution	Use --no-cache flag while building

Related Log Evidence

Log ID	Timestamp	Level	Message	Error Code
N/A	N/A	N/A	No direct log evidence found in logs CSV for this ticket.	N/A

Procedure

1. Confirm the image tag, expected change, actual behavior, and build environment.
2. Review Docker build logs to identify cached layers and unchanged instructions.
3. Check whether dependency files changed before application source files in the Dockerfile.
4. Rebuild using --no-cache to verify whether stale layers are causing the issue.
5. Clear builder cache only when approved and required.
6. Reorder Dockerfile instructions to cache dependencies safely while rebuilding changed code.
7. Use unique image tags and confirm image digest after build.
8. Deploy the rebuilt image and validate the expected change is present.

Common Commands

```
docker ps -a
docker logs <container-name-or-id> --tail=100
docker inspect <container-name-or-id>
```

```
docker stats --no-stream
docker images
docker network ls && docker network inspect <network-name>
docker volume ls && docker volume inspect <volume-name>
docker system df
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Old code in image	Cached COPY layer	Rebuild with --no-cache or change context
Old dependencies used	Dependency file layer cached	Update dependency file and rebuild
Wrong image deployed	Tag reused	Use unique tag or digest
Build slow after fix	Cache fully disabled	Reorder Dockerfile for safe caching

Best Practices

1. Use immutable tags in CI/CD.
2. Place dependency files before app source for controlled caching.
3. Verify image digest after build.
4. Avoid using latest tag in production.

References

Docker Docs | Internal Tickets CSV | Internal Logs CSV | Container Runtime Logs | Monitoring Dashboards | DevOps Runbook